

Sea Lyon Media Manager — macOS Apple Silicon Port Report

Source branch analyzed: LMM-DEV (commit [ec78b84](#))

Scope: Apple Silicon (arm64) exclusive. Intel Mac support not in scope.

Audience: Project owner / lead engineer.

Date: 2026-05-27

1. Executive Summary

The codebase is ~**85% portable** to macOS Apple Silicon today. It is already a Python 3.11 + PySide6 (Qt 6.11) application, which gives you a true cross-platform Qt foundation. The remaining 15% is concentrated in five clearly-bounded surfaces:

1. **Visual identity / QSS theming** — the UI looks "wrong" on macOS because of one hardcoded font in the stylesheet, not because Qt rendered it badly.
2. **Native runtime discovery** — libVLC, ffmpeg, fpcalc, libdiscid paths.
3. **Optical disc subsystem** — pure Win32 ctypes; not portable.
4. **Keyboard shortcuts and menu bar conventions** — Ctrl vs Cmd (⌘), native menu bar.
5. **Build & distribution** — PyInstaller spec, Inno Setup, GitHub Actions are 100% Windows.

Recommendation: Single codebase with `sys.platform` branching. Do NOT fork.

The platform-specific code is already well-isolated behind 25 discrete `sys.platform` checks, and the platform-dependent surface (disc subsystem, binary discovery, settings paths, build) totals roughly **8 files**. The hardcoded macOS font already exists in `app.py` — the architecture clearly anticipated multi-platform support. A fork would force you to manually back-port every Windows feature (which you said is "perfect") and double release work. A single tree is cheaper to maintain and easier to test.

The user-perceived macOS UI/UX problems trace to **one stylesheet line** (`lyon/ui/styles.py:13`) plus shortcut conventions and window-mode defaults — all fixable inside the single tree.

Sibling branch note: there is already a `LMM-MACOS` branch in this repository with a `PORT_PLAN_MACOS_ARM64.md` document and 47 lines changed in `build/lyon.spec`. This report was written from LMM-DEV per your instruction; the existing macOS branch should be reconciled into the recommendations here before more work diverges.

2. Architecture Snapshot

Layer	Tech	Cross-platform?
Language	Python 3.11	✓
GUI toolkit	PySide6 / Qt 6.11 (Essentials)	✓
Media engine	libVLC (python-vlc 3.0.21203)	✓ runtime, ✗ discovery
Tag I/O	mutagen 1.47	✓
Metadata	musicbrainzngs, discid (Win-only pin)	partial
Fingerprint	Chromaprint fpcalc binary + pyacoustid	✓ on PATH
Audio transcode / disc rip	ffmpeg binary + Windows raw CDDA reader	partial
Library watcher	watchdog 6.0	✓
Online	requests, defusedxml, yt-dlp	✓
Packaging	PyInstaller 6.20 + Inno Setup 6	✗ Windows only
Updater	Sparkle-style appcast (XML)	architecturally ✓, content ✗

The Python/Qt layer is fully portable. The Windows lock-in is in three places: optical disc handling, binary discovery for libVLC, and packaging.

Repository layout (relevant parts)

```

lyon/
  app.py          # entry point – already has darwin font branch
  ui/             # 40 PySide6 widget modules; UI bug surface is here
    styles.py     # *** central QSS – hardcoded Segoe UI ***
    main_window.py # tab bar, menu, shortcuts
    video_player_view.py # libVLC native handle attach (already cross-platform)
  core/
    settings.py   # app_data_dir() – wrong on macOS
    playback_backend.py # libVLC instance – Windows-only runtime config
    media_keys.py # already has a macOS NSEvent branch
    cd_detect.py  # pure Win32 ctypes; not portable
    ripper.py     # ffmpeg + Win32 raw CDDA fallback
    disc_playback.py # Windows drive-letter logic
    ffmpeg.py     # shutil.which – already portable
    fingerprint.py # fpcalc – already portable
    updater.py    # appcast parser; .exe URLs
    dlina_server.py # SSDP multicast – needs macOS sandbox test
  build/
    lyon.spec     # PyInstaller – Win-centric defaults
    lyon.iss      # Inno Setup – Windows-only
  scripts/
    build-windows.ps1 # PowerShell
  .github/workflows/
    windows-build.yml # runs-on: windows-latest only

```

3. Findings (Concrete, with File:Line Citations)

Severity legend: **C** = critical (port blocked or product broken), **H** = high (degraded UX or visible defect), **M** = medium, **L** = low (polish).

3.1 UI / UX — the actual reason it looks wrong on macOS

F-UI-1 (C) — Universal QSS forces Segoe UI everywhere, overriding the platform font set in `app.py`

`lyon/ui/styles.py:13:`

```
* { color: #eef7ff; font-family: "Segoe UI", "Tahoma", sans-serif; font-size: 9pt; }
```

The CSS universal selector wins over `QApplication.setFont(QFont("SF Pro Text", 13))` set at `lyon/app.py:115-116`. This is the **single biggest reason the UI feels off on Mac** — every label, button, tab, and menu falls back to Tahoma/sans-serif because Mac doesn't have Segoe UI. It also pins the size at 9pt, which is too small for macOS's intended visual density.

Fix: Make the family fall through to the system font, and let `app.py` own the size:

```
* { color: #eef7ff; font-family: "Segoe UI", -apple-system, "SF Pro Text", "Helvetica Neue", "Tahoma", sans-serif; }
```

Then remove the global `font-size: 9pt;` and bump the application font on macOS to `QFont(".AppleSystemUIFont", 13)` or `QGuiApplication.font()`.

F-UI-2 (C) — Keyboard shortcuts hardcoded to "Ctrl+..." with visible Ctrl labels

`lyon/ui/main_window.py:104-113:`

```
_TAB_SHORTCUTS = {  
    "Library": "Ctrl+1", "Now Playing": "Ctrl+2", "Radio": "Ctrl+3",  
    "Video": "Ctrl+4", "Disc": "Ctrl+5", "Rip": "Ctrl+6",  
    "YouTube": "Ctrl+7", "Podcasts": "Ctrl+8",  
}
```

Qt's `QKeySequence("Ctrl+1")` does silently re-map `Ctrl` → `⌘` on macOS when invoked, **but**:

- Tooltips and the Knowledge Base (`lyon/ui/knowledge_base_dialog.py:69-1430`) display the literal string "Ctrl+1" through "Ctrl+8", "Ctrl+Q", etc.
- The user sees "Ctrl" everywhere on macOS even though the shortcut needs `⌘`.

Fix: Two changes:

1. Bind shortcuts via `QKeySequence.StandardKey` where one exists; for tab navigation, store the canonical sequence and display the keys via `seq.toString(QKeySequence.NativeText)` which yields `⌘1` on macOS, `Ctrl+1` on Windows.
2. In `knowledge_base_dialog.py`, replace literal `<kbd>Ctrl+N</kbd>` strings with platform-aware rendering at view time (helper function returning `⌘` or `Ctrl` per `sys.platform`).

F-UI-3 (H) — No native menu bar; full functionality lives in custom tab bar

`lyon/ui/main_window.py:375-426` builds File/Playback/View/Settings/Help via `QMainWindow.menuBar()`. On macOS Qt does promote this to the screen-top menu bar by default — that part is fine — but the primary navigation (Library/Now Playing/Radio/Video/Disc/Rip/YouTube/Podcasts) is a custom `QTabBar` in a `QWidget#headerBar` (`main_window.py:200-211`). On macOS users expect:

- A toolbar or sidebar for primary destinations, not a flat tab strip styled with Windows Media Player gradients.
- `Cmd+number` shortcuts visible in the **View** menu (currently only in tooltips).

Fix: Add a "View → Go To ..." submenu mirroring the tab list with proper shortcuts. The tab bar can stay (cross-platform UI), but the menu bar should advertise every navigation target so macOS-style keyboard navigation works.

F-UI-4 (H) — Window opens maximized; macOS convention is zoomed/sized

lyon/app.py:126:

```
win.showMaximized()
```

On macOS, "maximize" doesn't exist as a concept — apps open at a saved size or a sensible default. `showMaximized()` produces a window that fills the screen below the menu bar and feels foreign. Mac apps either:

- Restore last size/position (preferred), or
- Open at the default size set by `resize(1100, 720)` already at `main_window.py:189`.

Fix:

```
if sys.platform == "darwin":
    win.show()
else:
    win.showMaximized()
```

Plus persist last-window geometry through QSettings or the existing `Settings` dataclass.

F-UI-5 (H) — Deep dark blue gradient is ambient on Windows, fights macOS

lyon/ui/styles.py is 626 lines of Windows-Media-Player-inspired graphite + cyan gradients. The QSS is aggressive: every `QPushButton`, `QTabBar::tab`, `QSlider::groove`, `QProgressBar::chunk`, etc. is overridden. macOS users expect the system to provide button chrome — heavy custom QSS makes the app look like an embedded web view.

Fix (low effort): Keep the dark theme but on macOS:

- Drop `font-family` from `*` so widgets inherit system font.
- Soften corner radii where macOS would round natively.
- Remove `min-height` overrides on buttons that conflict with Cocoa control sizing.

Fix (high effort): Ship a second, slimmer QSS variant gated on `sys.platform == "darwin"`. Same color palette, fewer pixel-perfect overrides.

F-UI-6 (M) — Tooltip palette is hardcoded for Windows High Contrast

lyon/ui/styles.py:108-117 forces black tooltip background with a white border:

```
QToolTip { background-color: #000000; color: #ffffff; border: 1px solid #ffffff; ... }
```

The comment explicitly says "Keep hover tooltips readable when Windows high contrast themes...". On macOS this style is visually loud; tooltips should follow the dark theme softly.

F-UI-7 (M) — Frameless OSD and fullscreen video may need macOS spaces handling

- `lyon/ui/osd.py:20` — frameless `Qt.FramelessWindowHint` for transient OSD overlay.
- `lyon/ui/video_player_view.py:258` — frameless fullscreen video window.

On macOS, frameless windows behave well but **don't enter Mission Control "fullscreen space" automatically**. If you want the video player to use the macOS green-button fullscreen, add `Qt.WindowFullScreen` state instead of frameless.

F-UI-8 (L) — Window icon may not appear in dock at expected resolution

`lyon/ui/branding.py` + `lyon/app.py:120-121` use `app_icon()`. Source PNG is `docs/brand/lyon-app-icon.png` (989 KB on LMM-DEV). On macOS the dock expects an `.icns` or at least a high-res square PNG. Currently the PyInstaller spec generates a Windows `.ico` at `build/lyon.spec:36-52`. **No `.icns` generation** for macOS — Apple's `iconutil` produces `.icns` from an `.iconset` directory.

3.2 Settings & Paths

F-FS-1 (C) — `app_data_dir()` uses `~/.config` on macOS instead of `~/Library/Application Support`

`lyon/core/settings.py:62-69`:

```
def app_data_dir() -> Path:
    if sys.platform == "win32":
        base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
    else:
        base = os.environ.get("XDG_CONFIG_HOME") or str(Path.home() / ".config")
    p = Path(base) / "LyonMusicManager"
    p.mkdir(parents=True, exist_ok=True)
    return p
```

This is the **only place writes happen on macOS** (`settings.json`, logs at `lyon/app.py:30`). Settings end up at `~/.config/LyonMusicManager/` instead of `~/Library/Application Support/LyonMusicManager/`. Functional today, but:

- Time Machine and iCloud Drive don't back up `~/.config` by default.
- The app is invisible to macOS users browsing `~/Library/Application Support`.
- Mac-style `defaults read` won't find it.

Fix:

```
def app_data_dir() -> Path:
    if sys.platform == "win32":
        base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
    elif sys.platform == "darwin":
        base = str(Path.home() / "Library" / "Application Support")
    else:
        base = os.environ.get("XDG_CONFIG_HOME") or str(Path.home() / ".config")
    p = Path(base) / "LyonMusicManager"
    p.mkdir(parents=True, exist_ok=True)
    return p
```

Add a one-shot migration: on first run on macOS, if `~/.config/LyonMusicManager` exists, move it to the new location.

F-FS-2 (L) — Default music root is correct on macOS

lyon/core/settings.py:54-59 falls back to `~/Music/Lyon` on non-Windows — that's the macOS convention. No change needed.

F-FS-3 (L) — File permissions

lyon/core/settings.py:503-507 `chmods` to `0o600`. Works on macOS, but Windows builds silently no-op (`chmod` has minimal effect on NTFS). No change needed.

3.3 libVLC Runtime — Will Fail to Load on macOS

F-VLC-1 (C) — `_configure_vlc_runtime_path()` only handles Windows

lyon/core/playback_backend.py:44-68:

```
def _configure_vlc_runtime_path() -> None:
    vlc_dir = bundled_bin_dir() / "vlc"
    if not vlc_dir.exists() or vlc_dir in _CONFIGURED_VLC_DIRS:
        return
    _prepend_path(vlc_dir)
    add_dll_directory = getattr(os, "add_dll_directory", None)  # Windows-only API
    if add_dll_directory is not None:
        ...
    plugins_dir = vlc_dir / "plugins"
    if plugins_dir.exists():
        os.environ.setdefault("VLC_PLUGIN_PATH", str(plugins_dir))
```

If no `bin/vlc/` exists at runtime (which it won't on a macOS install), this early-returns and the subsequent `importlib.import_module("vlc")` relies on `python-vlc`'s own libVLC discovery, which on macOS searches:

1. `LIBVLC_PYTHON_PATH` env var
2. `LD_LIBRARY_PATH`, `DYLD_LIBRARY_PATH`
3. `/Applications/VLC.app/Contents/MacOS/lib/libvlc.dylib`

...and **frequently fails** because Apple Silicon Macs ship VLC as an arm64 binary only since VLC 3.0.18, and `python-vlc` doesn't always find it without help.

Fix: Add a darwin branch that explicitly sets the dylib path:

```
elif sys.platform == "darwin":
    for candidate in (
        Path("/Applications/VLC.app/Contents/MacOS/lib"),
        Path("/opt/homebrew/lib"),  # Homebrew arm64
        Path("/usr/local/lib"),     # legacy
    ):
        if (candidate / "libvlc.dylib").exists():
            os.environ.setdefault("VLC_PLUGIN_PATH", str(candidate / "vlc" / "plugins"))
            os.environ.setdefault("DYLD_LIBRARY_PATH", str(candidate))
            break
```

Also gate the `add_dll_directory` block on `sys.platform == "win32"` for clarity.

F-VLC-2 (L) — Video output handle attach is already cross-platform

lyon/ui/video_player_view.py:856-864 correctly branches:

```
if sys.platform == "win32": self._player.set_hwnd(wid)
elif sys.platform.startswith("linux"): self._player.set_xwindow(wid)
else: self._player.set_nsobject(wid)
```

No change needed. This is a good pattern to mirror elsewhere.

F-VLC-3 (M) — `close_dll_handles()` is named after a Windows concept

`lyon/core/playback_backend.py:70-77` collects `_DLL_DIRECTORY_HANDLES` and releases them on shutdown. The list is unused on macOS, so this is a no-op, not a bug. Consider renaming to `release_runtime_handles()` to match cross-platform intent.

3.4 Optical Disc Subsystem — Cannot Run on macOS

F-DISC-1 (C-for-disc-features) — `cd_detect.py` is pure Win32 ctypes

`lyon/core/cd_detect.py:9-77` binds `kernel32.GetLogicalDrives`, `GetDriveTypeW`, `CreateFileW`, `DeviceIoControl` (`IOCTL_CDROM_READ_TOC_EX`), `winmm.mciSendStringW`. Every entry point (`list_cd_drives`, `has_audio_cd`, `read_disc_toc`, `eject_drive`, ...) opens with `if sys.platform != "win32": return []` or `return False`. Lines 40, 109, 127, 170, 222, 369.

Status: safe to import on macOS; **all CD features silently disabled**.

F-DISC-2 (H) — Ripper's raw-CDDA fallback is Windows-only

`lyon/core/ripper.py:389-517` defines `_WindowsCddaReader` (a ctypes `IOCTL_CDROM_RAW_READ` wrapper). The ripper uses it when ffmpeg lacks `libcdio` (`ripper.py:741-784`). On macOS, this fallback path is unavailable; **rip is only possible if ffmpeg was compiled with libcdio** (Homebrew ffmpeg does include it).

Status: if Homebrew ffmpeg is installed, ripping should work via `libcdio`. Verify at runtime with `_ffmpeg_supports_demuxer(ff, "libcdio")` at `ripper.py:738`. If not present, the code already emits a clear error.

F-DISC-3 (H) — `disc_playback.vlc_device()` builds Windows MRLs

`lyon/core/disc_playback.py:37-77` returns strings like `d:/` to feed to VLC's `cdda://` MRL. On macOS the device path is `/dev/disk2` (or `/dev/rdisk2` for raw access). VLC's CDDA MRL on macOS is `cdda:///dev/disk2`.

Decision needed: Modern Apple Silicon Macs **have no optical drive**. There hasn't been one in a Mac since 2013-2016 depending on model. You can choose:

- **Option A (recommended):** On macOS, hide the Disc tab and Rip tab entirely. Show in About: "Optical disc features are Windows-only."
- **Option B (more work):** Implement IOKit-based detection via PyObjC; expect <5% of macOS users to have a working external USB SuperDrive.

If you choose Option A, gate at `lyon/ui/main_window.py:103`:

```
_TAB_ORDER = ("Library", "Now Playing", "Podcasts", "Radio", "Video")
if sys.platform == "win32":
    _TAB_ORDER += ("Disc", "Rip")
```

```
_TAB_ORDER += ("YouTube",)
```

F-DISC-4 (M) — `ctdb_verify.py` and `discid` Python package

- `lyon/core/ctdb_verify.py:39` — `CREATE_NO_WINDOW` falls back to 0 on macOS, fine.
- `requirements.in:4` — `discid==1.3.0; sys_platform == "win32"` means the Python `discid` wrapper is only installed on Windows. The `cd_detect.py` import is already guarded.

Status: correctly gated. No change needed unless you decide to support disc features on Mac (Option B above).

3.5 Binary Discovery — Mostly OK

F-BIN-1 (L) — `ffmpeg` discovery is already portable

`lyon/core/ffmpeg.py:10-18:`

```
for name in ("ffmpeg.exe", "ffmpeg"):
    candidate = bin_dir / name
    if candidate.exists(): return candidate
found = shutil.which("ffmpeg.exe") or shutil.which("ffmpeg")
```

Works on macOS via `brew install ffmpeg` (lands at `/opt/homebrew/bin/ffmpeg`).

F-BIN-2 (L) — `fpcalc` discovery is already portable

`lyon/core/fingerprint.py:39,73` — `"fpcalc.exe" if sys.platform == "win32" else "fpcalc"`.

Works via `brew install chromaprint`.

F-BIN-3 (M) — No bundled macOS binaries

The Windows CI fetches `ffmpeg`, `fpcalc`, `libdiscid`, `libVLC` and bundles them in `bin/`. For a self-contained `.app` you need the macOS equivalents:

- `ffmpeg` (universal/arm64, static build from <https://evermeet.cx/ffmpeg/>)
- `fpcalc` (Chromaprint macOS release — currently not pinned in the CI)
- `libVLC`: ship as `VLC.app` bundled inside `Sea Lyon Media Manager.app/Contents/Frameworks/`, or require the user to install VLC.app separately (more common for Mac users).

3.6 Media Keys — Already Partially Ported, Off the Release Surface

`lyon/core/media_keys.py:1-12` includes this comment:

```
The macOS path (PyObjC ``NSEvent`` global monitor) remains for developers
running from source, but is not part of the advertised v1.0 product
surface. Treat it as best-effort.
```

`media_keys.py:64-98` implements `_register_macos_media_key_handler` using `AppKit.NSEvent.addGlobalMonitorForEventsMatchingMask_handler_` and decodes `NSSystemDefined` subtype 8 (media key event), key codes 16/17/18.

Status: functional, but:

1. `requirements.in` does **not** include `pyobjc-framework-Cocoa`. The `import AppKit` will fail silently on a fresh `pip install -r requirements.in` and the handler will return `None`.
2. macOS Big Sur+ requires the app to be granted **Input Monitoring** and possibly **Accessibility** permission for global media key capture. The app does not detect or prompt for this.

Fix: Add `pyobjc-framework-Cocoa` to `requirements.in` under a `sys_platform == "darwin"` marker. Plumb a settings toggle that asks the user to grant permission.

3.7 Updater — Single-Platform Appcast

F-UPD-1 (H) — Appcast XML hard-wires `.exe` installer URLs

`scripts/generate-appcast.py` and `lyon/core/updater.py` emit an `<enclosure>` pointing at `SealYonMediaManager-${ver}-Setup.exe`. There is no platform discrimination.

Fix: Sparkle's spec supports per-OS enclosures:

```
<enclosure url="...-Setup.exe"    sparkle:os="windows" length="..." type="application/octet-stream"/>
<enclosure url="...-arm64.dmg"    sparkle:os="macos"    length="..." type="application/octet-stream"
    sparkle:minimumSystemVersion="11.0"/>
```

On the client side (`lyon/core/updater.py`), filter enclosures by `sys.platform` before presenting the update.

F-UPD-2 (M) — Updater dialog text assumes Windows installer

`lyon/ui/update_dialog.py` and `core/updater.py` route the user to download the installer and run it. On macOS the experience should be: download `.dmg`, mount, drag-to-Applications. The user-visible copy needs platform variants.

3.8 DLNA / Cast — Likely Works, Needs Sandbox Check

F-NET-1 (M) — Multicast may need entitlements under Hardened Runtime

`lyon/core/dlna_server.py` and `dlna_renderer_discovery.py` use SSDP (239.255.255.250:1900). When the macOS `.app` is built with the Hardened Runtime (required for notarization), multicast and local-network access need:

- `com.apple.security.network.client` entitlement
- `com.apple.security.network.server` entitlement
- `NSLocalNetworkUsageDescription` in `Info.plist`
- macOS will prompt the user the first time the app tries to discover devices.

`pychromecast` (used by `cast_controller.py`) is fine across platforms.

F-NET-2 (L) — Cast UI assumes named local networks

No bug, but on macOS the local-network privacy prompt is unmistakable and disruptive. Time the first SSDP discovery to a user action (clicking "Cast") rather than app startup.

3.9 Subprocess Window Suppression — Already Correct

`lyon/core/ripper.py:72`, `lyon/core/replaygain.py:19`, `lyon/core/ctdb_verify.py:39` all use:

```
_NO_WINDOW = subprocess.CREATE_NO_WINDOW if sys.platform == "win32" else 0
```

This pattern is correct and needs no change.

3.10 Build & Distribution — Hardest Single Workstream

F-BUILD-1 (C) — PyInstaller spec assumes Windows

`build/lyon.spec`:

- `STRIP_BINARIES = sys.platform != "win32"` (line 17) — already cross-platform aware.
- The `bin/` discovery loop (lines 54-62) doesn't differentiate platforms; on macOS you'd bundle different binaries.
- `EXE(... console=False, icon=icon_file)` (lines 128-140) — icon is a `.ico`; macOS needs `.icns`.
- No `BUNDLE(...)` target — needed to produce a `Sea Lyon Media Manager.app` directory.

Fix: Add a darwin branch at the end of the spec:

```
if sys.platform == "darwin":
    app = BUNDLE(
        coll,
        name="Sea Lyon Media Manager.app",
        icon=str(ROOT / "build" / "lyon-app-icon.icns"),
        bundle_identifier="com.drshoctopus.sealyonmediamanager",
        info_plist={
            "CFBundleShortVersionString": __version__,
            "CFBundleVersion": __version__,
            "LSMinimumSystemVersion": "11.0",
            "NSHighResolutionCapable": True,
            "NSLocalNetworkUsageDescription": "Sea Lyon discovers Chromecast and DLNA renderers
on your local network.",
            "NSAppleEventsUsageDescription": "Used for media key handling.",
            "LSApplicationCategoryType": "public.app-category.music",
        },
    )
```

F-BUILD-2 (C) — Inno Setup is Windows-only

`build/lyon.iss` cannot be reused. macOS distribution path:

1. PyInstaller `.app` bundle (above).
2. Codesign with Developer ID Application certificate.
3. Notarize via `notarytool`.
4. Staple ticket.

5. Package into a `.dmg` (use `create-dmg` or `dmgbuidl`).

F-BUILD-3 (C) — CI is Windows-only

`.github/workflows/windows-build.yml` is the only build workflow. It hard-codes `runs-on: windows-latest`, fetches Windows binaries from gyan.dev, acoustid, metabrainz, and videolan, and writes a Windows installer.

Fix: Add `.github/workflows/macos-build.yml` running on `macos-14` (which is Apple Silicon). High-level steps:

1. `actions/checkout@v4`
2. `actions/setup-python@v5` with 3.11 arm64
3. `brew install ffmpeg chromaprint libdiscid` (cached)
4. `pip install -r requirements-build.txt`
5. Run pytest (test floor as in Windows workflow)
6. `iconutil -c icns build/icon.iconset -o build/lyon-app-icon.icns`
7. `pyinstaller build/lyon.spec`
8. Codesign: `codesign --deep --options runtime --entitlements build/lyon.entitlements --sign "$DEVELOPER_ID" dist/Sea\ Lyon\ Media\ Manager.app`
9. Notarize: `xcrun notarytool submit --wait ...`
10. Staple: `xcrun stapler staple ...`
11. `create-dmg` → `SeaLyonMediaManager-${ver}-arm64.dmg`
12. Upload artifact.

Secrets needed: `APPLE_DEVELOPER_ID_CERT`, `APPLE_DEVELOPER_ID_CERT_PASSWORD`, `APPLE_ID`, `APPLE_TEAM_ID`, `APPLE_APP_PASSWORD` (app-specific).

F-BUILD-4 (M) — `requirements-build.txt` may not be macOS-friendly

`requirements-build.in` pins `pyinstaller==6.20.0` and `Pillow==12.2.0`. Both fine on arm64. Verify `requirements.txt` lock works on macOS by running `pip-compile --platform macosx_11_0_arm64` or checking the existing hashes.

3.11 Tests

`tests/` runs in offscreen mode on Windows CI (`windows-build.yml:48` sets `QT_QPA_PLATFORM=offscreen`). The test floor is 680 tests. On macOS:

- Most tests should pass identically.
- Disc-related tests likely skip on non-Windows (verify).
- Any test that touches `app_data_dir()` or `bundled_bin_dir()` must be platform-aware.

Add a macOS pytest job to CI before shipping.

4. Branch Strategy: Why a Single Codebase Wins

Option A — Single tree with `sys.platform` branches (RECOMMENDED)

Pros

- Existing code already follows this pattern: 25 `sys.platform` checks across 14 files. The architecture anticipates it.
- One bug fix benefits all platforms. The user said Windows UX is perfect; you want to keep it perfect without back-porting.
- The platform-specific surface is small and well-contained: 5–8 files have meaningful platform code.
- One CHANGELOG, one version, one tag triggers both builds.
- Shared CI for the tests; only the packaging steps split.
- macOS bug fixes that involve Qt or PySide6 often help Windows too (and vice versa).

Cons

- More conditional code in core modules.
- Reviewers must keep both platforms in mind on every PR.
- Slightly larger source tree (entitlements, .icns, .iss, .ps1 all coexist).

Option B — Hard fork (`LMM-MACOS-DEV` branch as the macOS line)

Pros

- Aggressive Mac-only refactors (e.g. ripping out the disc subsystem entirely) become easier.
- macOS-specific dependencies live in their own requirements file without markers.

Cons

- **Every Windows feature must be manually re-applied to the macOS branch** indefinitely. You described Windows UI/UX as "perfect" — preserving that drift-free is meaningful ongoing work.
- Double release cadence, double tagging, double appcast logic.
- The shared 85% (UI, library, scrobbling, YouTube, podcasts, radio, equalizer) duplicates rot.
- Merges from `LMM-DEV` into `LMM-MACOS-DEV` will get progressively harder.

Option C — Plugin/extras architecture

Restructure so disc / Windows-only code is an installable extra (e.g. `pip install sealyon[disc]` installs `discid` and the Win32 ctypes glue). Long-term clean but a large refactor for marginal benefit unless you plan a Linux port too.

My recommendation

Adopt Option A. Specifically:

1. Merge `LMM-MACOS` (the existing branch with the port plan) back into `LMM-DEV` once the immediate critical fixes land, then delete `LMM-MACOS`.
2. Use `LMM-DEV` as the single development line going forward.
3. Add release tags that trigger **both** the Windows workflow and a new macOS workflow in parallel — both produce platform-specific artifacts attached to the same GitHub Release.
4. Keep an `LMM-MASTER` (or `main`) that is the released-stable branch, same for both platforms.

This matches what the codebase is already doing (per-platform branches in `app.py`, `media_keys.py`, `playback_backend.py`, `settings.py`). You'd be finishing the existing pattern, not replacing it.

5. Phased Porting Plan

Phase 0 — Foundations (1 day)

- `settings.py`: macOS `app_data_dir()` → `~/Library/Application Support/LyonMusicManager` + migration from `~/.config/LyonMusicManager` (F-FS-1).
- `requirements.in`: add `pyobjc-framework-Cocoa`; `sys_platform == "darwin"` (for F-MEDIA, F-BUILD entitlements).
- Document Homebrew prerequisites: `brew install ffmpeg chromaprint libvlc` (and tell user to install VLC.app from videolan.org as an alternative).

Phase 1 — Runtime fixes (2 days)

- `playback_backend.py`: darwin branch for libVLC discovery (F-VLC-1).
- `cd_detect.py` / `ripper.py` / `disc_playback.py`: confirm graceful no-op on macOS; if shipping disc support, scope IOKit work as a separate epic.
- Hide Disc and Rip tabs on macOS (or show with "Windows only" message) — `main_window.py:103` (F-DISC-3).

Phase 2 — UI parity (3-5 days)

- `styles.py:13`: replace `font-family: "Segoe UI"...` with system-fallback stack; drop universal font-size (F-UI-1).
- `app.py:115`: use `QFont(".AppleSystemUIFont", 13)` or rely on system default.
- `app.py:126`: don't `showMaximized()` on macOS (F-UI-4).
- `main_window.py:104-113` + `knowledge_base_dialog.py`: render shortcut labels via `QKeySequence.NativeText` (F-UI-2).
- Smoke-test every dialog on a Retina display; capture screenshots; iterate on QSS until each view matches the Windows feel (F-UI-5).
- Test menu bar — Qt should auto-promote to screen menu bar, verify it does (F-UI-3).

Phase 3 — Build & distribute (3 days)

- Generate `.icns` from `docs/brand/lyon-app-icon.png` (use `iconutil` in CI).
- `build/lyon.spec`: add `BUNDLE()` block with `Info.plist` + entitlements (F-BUILD-1).

- ❑ Write `build/lyon.entitlements` with Hardened Runtime entitlements (network client/server, JIT denied, library validation).
- ❑ New `.github/workflows/macos-build.yml` running on `macos-14` (arm64). Includes pytest, codesign, notarize, staple, dmg (F-BUILD-3).
- ❑ Add Apple Developer secrets to repository settings.

Phase 4 — Updater (1 day)

- ❑ `scripts/generate-appcast.py`: emit both Windows and macOS `<enclosure>` elements, each with `sparkle:os` attribute (F-UPD-1).
- ❑ `lyon/core/updater.py`: filter by `sys.platform` when reading the feed (F-UPD-1).
- ❑ `lyon/ui/update_dialog.py`: platform-aware copy "Download installer" vs "Download disk image" (F-UPD-2).

Phase 5 — Polish (2-3 days)

- ❑ Media keys: `_register_macos_media_key_handler` warns about Input Monitoring permission on first use (F-MEDIA).
- ❑ DLNA: trigger first SSDP discovery on user click, not boot, to defer local-network prompt (F-NET-1).
- ❑ Run on Apple Silicon hardware: verify VLC playback, fingerprint matching, scrobbler, library scan, YouTube DL.
- ❑ Tag and release `v1.x.0` with macOS DMG artifact.

Total estimate: 12-16 engineer-days for a release-quality macOS Apple Silicon build, assuming Windows codebase remains in its current shape.

6. Concrete Code Changes — Quick Reference

File	Lines	Change	Phase
lyon/core/settings.py	62-69	Add <code>elif sys.platform == "darwin"</code> for Application Support	0
lyon/core/playback_backend.py	44-68	Add darwin libVLC dylib + plugin-path resolution	1
lyon/ui/styles.py	13	Remove <code>font-family</code> from <code>*</code> , or add SF Pro fallback	2
lyon/app.py	115-116	Switch to <code>.AppleSystemUIFont</code> and verify size	2
lyon/app.py	126	macOS: <code>win.show()</code> not <code>showMaximized()</code>	2
lyon/ui/main_window.py	103	Conditionally drop Disc/Rip tabs on macOS	1
lyon/ui/main_window.py	215, 334+	Translate shortcut labels via <code>NativeText</code>	2
lyon/ui/knowledge_base_dialog.py	many	Replace literal <code><kbd>Ctrl+X</kbd></code> with helper	2
requirements.in	4-6	Add <code>pyobjc-framework-Cocoa</code> ; <code>sys_platform == "darwin"</code>	0
build/lyon.spec	128+	Append <code>BUNDLE(...)</code> for darwin	3
scripts/generate-appcast.py	(all)	Emit per-OS enclosures	4
.github/workflows/	new	<code>macos-build.yml</code> workflow	3

7. Risks & Open Questions

- VLC bundling vs requiring VLC.app.** Bundling libVLC inside the `.app` is more reliable but blows up the DMG size (~120 MB). Requiring users to install VLC.app first is the macOS-norm but adds friction. Recommendation: require VLC.app, link to videolan.org from the About dialog, and detect-and-warn if missing.
- Code signing certificate.** Notarization requires an Apple Developer ID Application certificate (\$99/year). Without it, users see a Gatekeeper warning and must right-click → Open. Plan for the \$99/year line item.
- Optical features.** You should explicitly decide before Phase 1: do macOS users see Disc/Rip tabs at all? Recommend hiding (Option A above). Hiding is one-line in `_TAB_ORDER`.
- HiDPI / Retina.** `QApplication.setHighDpiScaleFactorRoundingPolicy(PassThrough)` at `app.py:107-109` is correct, but several hardcoded pixel values in QSS (e.g. `min-height: 32px`, `width: 12px` for scrollbars) may feel off on Retina. Visual QA pass needed.

5. **Hardened Runtime + python-vlc.** Some Python C extensions are stripped by codesign's `--options runtime`. Test extensively after notarization; you may need `--entitlements` allowing dyld interposing.
 6. **Existing LMM-MACOS branch.** Diff (`git diff --stat LMM-DEV..LMM-MACOS`) shows extensive deletions of CHANGELOG, EULA, RELEASE_*, and additions of `PORT_PLAN_MACOS_ARM64.md`, `lyon-installer-wizard.png` removed, etc. Resolve whether to merge that branch's intent or discard it before starting fresh work.
-

8. Final Recommendation

Use a single codebase on LMM-DEV. The platform-specific code is already isolated and tracked. The macOS port is a 12-16 day effort dominated by:

1. **Three lines of stylesheet** (the font hack) that single-handedly explain most of the visual difference you're seeing.
2. **One settings path** that should point at `~/Library/Application Support`.
3. **One libVLC discovery branch** to make playback work.
4. **A new CI workflow** producing a notarized DMG.

Everything else — disc handling, custom QSS theming, menu polish — is incremental refinement on top of those four anchors. There's no architectural reason to fork.

End of report.